



TRAVAUX DIRIGÉS – SÉRIE 2

Algorithmique et Programmation

Graphes, géométrie, combinatoire et programmation dynamique

Ce document regroupe une seconde série de problèmes algorithmiques issus de compétitions de programmation. Pour chaque exercice, une formulation détaillée est donnée avec le format d'entrée/sortie attendu.

1 All Your Base

Exercice 1 — All Your Base

En l'an 2100, des extraterrestres sont arrivés sur Terre. À côté d'un message dans un langage cryptique, ils ont écrit une série de symboles. Les scientifiques en ont conclu que ces symboles représentent un nombre : le nombre de secondes avant le début d'une guerre.

Malheureusement, on ne sait pas ce que chaque symbole signifie. On a déduit que chaque symbole représente un chiffre, mais on ne connaît ni la correspondance entre symboles et chiffres, ni la base utilisée par les extraterrestres.

Ce que l'on sait :

- Le nombre est positif.
- Le premier symbole ne représente pas le chiffre zéro.
- La base est au moins 2 (pas de base unaire).

Objectif : Déterminer la **valeur minimale** possible du nombre représenté par la séquence de symboles.

Exemple : Si la séquence est `ab2ac999` :

- En base 10 avec $a = 3, b = 1, 2 = 5, c = 3, 9 = 6$, cela pourrait valoir 31 536 000 (un an exactement).

- En base 6 avec une autre correspondance, cela pourrait valoir 398 951.
- On cherche la plus petite valeur possible.

Format d'entrée / sortie

Entrée : T cas de test. Chaque cas est une chaîne de symboles (lettres minuscules et chiffres).

Sortie : Pour chaque cas, la valeur minimale possible.

2 Crazy Rows

Exercice 2 — Crazy Rows

On vous donne une matrice $N \times N$ composée de 0 et de 1. Vous pouvez **échanger deux lignes adjacentes** de la matrice.

Objectif : Obtenir une matrice **triangulaire inférieure**, c'est-à-dire telle que pour chaque ligne X ($1 \leq X \leq N$), il n'y a aucun 1 dans les colonnes strictement à droite de la colonne X .

Formellement, pour la ligne X , tout élément en position (X, j) avec $j > X$ doit être 0.

Question : Quel est le nombre **minimum** d'échanges de lignes adjacentes nécessaires ?

Format d'entrée / sortie

Entrée : T cas de test. Chaque cas : un entier N , suivi de N lignes de N bits.

Sortie : Pour chaque cas, le nombre minimal d'échanges.

Exemple : Pour la matrice $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$, il faut 1 échange pour obtenir $\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$.

3 Soudoyer les prisonniers

Exercice 3 — Bribe the Prisoners

Dans un royaume, P cellules de prison sont disposées en ligne, numérotées de 1 à P . Les cellules i et $i + 1$ sont adjacentes et les prisonniers communiquent entre voisins par des fenêtres.

Tous les prisonniers vivent en paix jusqu'à ce qu'un prisonnier soit libéré. Lorsqu'un prisonnier est libéré en cellule A , l'information se propage de proche en proche dans les deux directions depuis A , jusqu'à atteindre :

- la cellule 1 ou P , ou
- une cellule déjà vide.

Chaque prisonnier qui apprend la libération d'un autre devient furieux et casse tout dans sa cellule, **sauf s'il reçoit une pièce d'or** en pot-de-vin. Les pots-de-vin n'ont d'effet

que pour un jour.

Objectif : Étant donné la liste des Q prisonniers à libérer (un par jour, dans un ordre que vous choisissez), minimiser le nombre total de pièces d'or dépensées.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : P (nombre de cellules) et Q (nombre de prisonniers à libérer), suivis des Q numéros de cellules.

Sortie : Le nombre minimum de pièces d'or.

4 Le prochain nombre

Exercice 4 — The Next Number

Vous écrivez une liste de nombres en ordre croissant. Cette liste contient tous les nombres dont la représentation décimale contient exactement D_i occurrences du chiffre i , pour chaque i de 1 à 9. Le chiffre 0 peut apparaître n'importe quel nombre de fois (y compris aucune).

Exemple : Si la liste contient tous les nombres avec exactement deux 1 et un 5, elle commence par : 115, 151, 511, 1015, 1051, ...

Objectif : Étant donné N , le dernier nombre que vous avez écrit, déterminer le **suivant** dans la liste.

Format d'entrée / sortie

Entrée : T cas de test. Chaque cas est un entier N .

Sortie : Le prochain nombre de la liste après N .

5 Centre de masse

Exercice 5 — Center of Mass

Vous étudiez un essaim de N lucioles. Chaque luciole se déplace en ligne droite à vitesse constante dans l'espace 3D. Vous êtes positionné à l'origine $(0, 0, 0)$.

Pour chaque luciole i , vous connaissez :

- sa position initiale (x_i, y_i, z_i) à $t = 0$,
- sa vitesse (vx_i, vy_i, vz_i) .

La position de la luciole i au temps t est donc :

$$P_i(t) = (x_i + vx_i \cdot t, y_i + vy_i \cdot t, z_i + vz_i \cdot t)$$

Le **centre de masse** de l'essaim au temps t est :

$$M(t) = \frac{1}{N} \sum_{i=1}^N P_i(t)$$

Soit $d(t) = \|M(t)\|$ la distance entre le centre de masse et l'origine. Trouvez :

- $d_{\min}$: la valeur minimale de $d(t)$ pour $t \geq 0$,
- $t_{\min}$: le plus petit t tel que $d(t) = d_{\min}$.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : N , puis N lignes avec $x_i, y_i, z_i, vx_i, vy_i, vz_i$.

Sortie : $d_{\min}$ et $t_{\min}$ pour chaque cas, avec au moins 6 décimales.

6 Stock Charts

Exercice 6 — Stock Charts

Vous rédigez le résumé économique de fin d'année et souhaitez afficher les courbes de prix de n actions sur k instants.

Un *simple chart* relie les points $(0, p_0), (1, p_1), \dots, (k-1, p_{k-1})$ d'une action. Un *overlaid chart* superpose plusieurs *simple charts* sur le même graphique. Pour éviter toute confusion, **les courbes d'un même overlaid chart ne doivent jamais se croiser ni se toucher**.

Formellement, deux actions A et B peuvent être superposées si et seulement si :

$$\forall j \in \{0, \dots, k-1\}, A_j < B_j \quad \text{ou} \quad \forall j, A_j > B_j$$

(l'une domine strictement l'autre à chaque instant).

Objectif : Déterminer le nombre **minimum** d'*overlaid charts* nécessaires pour afficher toutes les actions.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : n et k , puis n lignes de k prix.

Sortie : Le nombre minimum d'*overlaid charts*.

7 Arrosage des plantes

Exercice 7 — Watering Plants

Dans votre serre, vous avez n plantes à arroser. Chaque plante occupe une zone circulaire de centre (x_i, y_i) et de rayon r_i . Aucune plante ne chevauche ou ne touche une autre.

Vous allez acheter **deux arroseurs**, chacun couvrant un disque de rayon R . L'un fonctionne le matin, l'autre le soir. Pour qu'une plante soit suffisamment arrosée, son disque entier doit être contenu dans le disque d'**au moins un** des deux arroseurs.

Objectif : Trouver la valeur **minimale** de R telle qu'il est possible de placer les deux arroseurs pour arroser toutes les plantes.

Format d'entrée / sortie**Entrée :** T cas. Chaque cas : n , puis n lignes avec x_i, y_i, r_i .**Sortie :** Le rayon minimal R , avec au moins 6 décimales.

8 Arbre de décision

Exercice 8 — Decision Tree

Un **arbre de décision** est une structure de données permettant de classer des animaux en “mignons” ou non, en fonction de leurs *features*. Chaque nœud de l'arbre possède :

- un **poids** w (réel entre 0 et 1),
- optionnellement : un nom de *feature* et deux sous-arbres (gauche et droit).

Évaluation : On part de la racine avec une probabilité $p = 1$.

1. Multiplier p par le poids w du nœud courant.
2. Si le nœud est une feuille : p est la probabilité que l'animal soit mignon.
3. Sinon : si l'animal possède la *feature* du nœud, descendre dans le sous-arbre gauche ; sinon, descendre dans le sous-arbre droit.

Exemple : Pour un castor (features : **furry**, **freshwater**) :

Racine ($w = 0,2$, feature **furry**) → fils gauche ($w = 0,81$, feature **fast**) → fils droit ($w = 0,2$).

Résultat : $p = 1 \times 0,2 \times 0,81 \times 0,2 = 0,0324$.

Format d'entrée / sortie**Entrée :** T cas. Chaque cas contient l'arbre (format parenthésé récursif) suivi de A animaux avec leurs features.**Sortie :** Pour chaque animal, la probabilité d'être mignon (au moins 6 décimales).

9 Photo d'équipe de football

Exercice 9 — Football Team

Une équipe de football pose pour une photo. Chaque joueur est situé en (x, y) où y est le numéro de rangée et x la position horizontale. Les x sont tous distincts.

Pour rendre la photo plus esthétique, on impose une contrainte de **couleurs de maillot** : deux joueurs $P_1 = (x_1, y_1)$ et $P_2 = (x_2, y_2)$ avec $x_1 < x_2$ doivent avoir des couleurs différentes si :

1. $|y_1 - y_2| \leq 1$ (même rangée ou rangées adjacentes), **et**
2. il n'existe aucun joueur (x_3, y_2) avec $x_1 < x_3 < x_2$ (i.e., P_2 est le plus proche voisin

de droite de P_1 dans la rangée y_2).

Objectif : Trouver le nombre **minimum** de couleurs de maillot distinctes nécessaires.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : n (nombre de joueurs), puis n paires (x_i, y_i) .

Sortie : Le nombre minimal de couleurs.

10 Intervalles intéressants

Exercice 10 — Interesting Ranges

Un entier positif est un **palindrome** si sa représentation décimale (sans zéros de tête) se lit de la même façon à l'endroit et à l'envers. Par exemple : 5, 77, 363, 4884, 11111, 12121, 349943.

Un intervalle $[L, R]$ d'entiers est dit **intéressant** s'il contient un nombre **pair** de palindromes.

Objectif : Étant donné un intervalle $[L, R]$, déterminer combien de ses sous-intervalles $[L_1, R_1]$ (avec $L \leq L_1 \leq R_1 \leq R$) sont intéressants.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : deux entiers L et R .

Sortie : Le nombre de sous-intervalles intéressants.

Remarque : L'intervalle $[10, 13]$ contient un seul palindrome (11). Il y a 10 sous-intervalles au total ; ceux qui ne contiennent pas 11 (ou le contiennent un nombre pair de fois, ici 0) sont intéressants.

11 Grille doublement triée

Exercice 11 — Doubly-sorted Grid

Une grille rectangulaire remplie de lettres minuscules est dite **doublement triée** si :

- dans chaque **ligne**, les lettres sont en ordre non-décroissant de gauche à droite,
- dans chaque **colonne**, les lettres sont en ordre non-décroissant de haut en bas.

Exemples de grilles doublement triées :

abc	ace
def	ade
ghi	bdg

On vous donne une grille **partiellement remplie**. Certaines cellules contiennent une lettre, les autres sont vides.

Objectif : Compter le nombre de façons de remplir les cellules vides avec des lettres minuscules pour que la grille soit doublement triée. Le résultat doit être donné modulo 10 007.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : dimensions $R \times C$, puis la grille (avec 0 pour les cellules vides).

Sortie : Le nombre de façons modulo 10 007.

12 Billes (Marbles)

Exercice 12 — Marbles

On dispose de $2n$ billes sur une grille, placées aux coordonnées $(1, 0), (2, 0), \dots, (2n, 0)$. Les billes sont colorées en n couleurs, avec exactement 2 billes de chaque couleur. Pour chaque couleur, il faut tracer un **chemin** reliant les deux billes de même couleur. Chaque chemin est composé de segments horizontaux et verticaux entre points de la grille.

Contraintes :

- Aucun chemin ne peut croiser ou toucher un autre chemin.
- Aucun chemin ne peut franchir la ligne $y = 0$; il peut seulement la toucher aux positions de ses deux billes.
- Le premier et dernier segment de chaque chemin sont verticaux.

La **hauteur** d'une solution est la différence entre la coordonnée y maximale et minimale utilisées par l'ensemble des chemins.

Objectif : Trouver la hauteur **minimale**, ou -1 si aucune solution n'existe.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : n , puis $2n$ couleurs.

Sortie : La hauteur minimale ou -1 .

13 Tours Wi-Fi

Exercice 13 — Wi-fi Towers

On dispose d'un réseau de tours sans fil. Chaque tour i a une portée r_i et peut envoyer des données à toute tour distante de $\leq r_i$.

Toutes les tours utilisent le protocole A . Un nouveau protocole B , plus performant, est disponible. On envisage de migrer certaines tours vers B .

Contrainte de compatibilité : Si une tour T utilise le protocole B , alors **toute** tour à portée de T doit aussi utiliser B (pour comprendre les données envoyées par T).

L'inverse n'est pas requis : une tour en B peut recevoir des données de tours en A .
 Chaque tour a un **score** (positif ou négatif) représentant le bénéfice net de sa migration.
Objectif : Choisir l'ensemble de tours à migrer de sorte que :

1. La contrainte de compatibilité est respectée.
2. La **somme des scores** est maximale.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : n tours avec positions, portées et scores.

Sortie : Le score total maximal.

14 Lumières et ombres

Exercice 14 — Lights

Dans une grande salle carrée, il y a deux sources de lumière ponctuelles : l'une **rouge** et l'autre **verte**. Il y a aussi n **piliers** circulaires qui bloquent la lumière et projettent des ombres.

La lumière se propage en ligne droite et est absorbée par les murs et les piliers. En chaque point de la salle (hors piliers), on observe une des quatre situations :

- **Noir** : aucune lumière n'atteint le point.
- **Rouge** : seule la lumière rouge atteint le point.
- **Vert** : seule la lumière verte atteint le point.
- **Jaune** : les deux lumières atteignent le point.

Objectif : Calculer l'aire totale de chacune des quatre zones (ne pas inclure l'aire des piliers).

Format d'entrée / sortie

Entrée : T cas. Chaque cas : positions des sources, nombre et paramètres des piliers, taille de la salle.

Sortie : Les quatre aires avec au moins 6 décimales.

15 Périmètre minimal

Exercice 15 — Min Perimeter

On vous donne un ensemble de n points à coordonnées entières dans le plan.

Objectif : Calculer le plus petit **périmètre** d'un triangle formé par trois points distincts de cet ensemble.

Le périmètre d'un triangle de sommets A, B, C est :

$$\text{Périmètre} = \|AB\| + \|BC\| + \|CA\|$$

où $\|PQ\| = \sqrt{(x_P - x_Q)^2 + (y_P - y_Q)^2}$ est la distance euclidienne.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : n , puis n lignes avec x_i, y_i (n pouvant être très grand).

Sortie : Le périmètre minimal (entier ou avec décimales selon la précision requise).

16 Square Math

Exercice 16 — Square Math

On considère un carré de $W \times W$ cellules (W^2 cellules au total). Chaque cellule contient :

- un chiffre de 0 à 9, ou
- un opérateur (+ ou -).

Le carré est dit **arithmétique** si :

- aucune paire de chiffres n'est adjacente (horizontalement ou verticalement),
- aucune paire d'opérateurs n'est adjacente.

Dans le puzzle **Square Math**, on part d'une cellule numérique et on se déplace horizontalement ou verticalement, une cellule à la fois, en terminant sur une cellule numérique. L'expression mathématique ainsi obtenue est évaluée. On peut revisiter les cellules.

Exemple : Dans le carré 3×3 :

$$\begin{array}{c} 2+3 \\ +4- \\ 1+0 \end{array}$$

En partant de 2, allant à droite (+), puis en bas (4), on obtient $2 + 4 = 6$.

Objectif : Pour chaque valeur cible donnée, trouver une expression (chemin) dans le carré qui l'évalue, ou indiquer qu'aucune n'existe.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : W , la grille, puis Q valeurs cibles.

Sortie : Pour chaque cible, un chemin solution ou **IMPOSSIBLE**.

17 Alphetomials

Exercice 17 — Alphetomials

On considère des polynômes multivariés de degré ≤ 4 , sur 26 variables (a à z). Chaque terme est un produit de lettres minuscules (avec éventuellement des répétitions). Les termes sont séparés par $+$.

Exemple : Le polynôme `aber+aab+c` a trois termes.

Évaluation sur une chaîne S : Chaque variable est remplacée par le nombre d'occurrences de la lettre correspondante dans S .

Exemple : Pour $S = \text{"abracadabra edgar"}$ et le polynôme `aber+aab+c` :

$a = 6, b = 2, e = 1, r = 3, c = 1$, donc $p(S) = 6 \times 2 \times 1 \times 3 + 6 \times 6 \times 2 + 1 = 36 + 72 + 1 = 109$.

Étant donné un dictionnaire de mots, une **d -phrase** est une chaîne formée de d mots du dictionnaire séparés par des espaces ($S = S_1 S_2 \dots S_d$). Pour chaque $1 \leq d \leq K$ ($K \leq 10$), calculer la somme de $p(S)$ sur toutes les d -phrases possibles, modulo 10 009.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : K , le polynôme, le nombre de mots et les mots du dictionnaire.

Sortie : K valeurs (les sommes pour $d = 1, 2, \dots, K$), modulo 10 009.

18 EZ-Sokoban

Exercice 18 — EZ-Sokoban

Le Sokoban est un célèbre jeu de puzzle japonais. Le but est de pousser des boîtes vers des emplacements cibles dans un entrepôt. Pour pousser une boîte, la case derrière vous et la case devant la boîte doivent être vides. On ne peut pousser qu'une boîte à la fois. Dans cette variante **EZ-Sokoban**, les boîtes contiennent des aimants puissants et doivent rester **connectées** (chaque boîte doit être accessible depuis toute autre en passant par des boîtes partageant un bord). Si un mouvement rompt la connexion, on entre en **mode dangereux** : le prochain mouvement **doit** rétablir la connexion.

Objectif : Étant donné la configuration de l'entrepôt (murs, boîtes, positions cibles, position du joueur), trouver la séquence de mouvements minimale pour placer toutes les boîtes sur les cibles, en respectant la contrainte de connexion.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : dimensions de la grille, puis la grille avec murs, boîtes, cibles et joueur.

Sortie : Le nombre minimum de mouvements, ou -1 si impossible.

19 Un problème de creusement

Exercice 19 — A Digging Problem

Une grotte est en feu et enfumée. Vous devez creuser votre chemin vers le bas pour respirer. La grotte est une matrice $R \times C$ composée de roche solide et de trous d'air. Vous démarrez en position $(1, 1)$ (coin supérieur gauche). À chaque étape :

- Vous pouvez vous déplacer d'une case vers la gauche ou la droite, si la case cible est un trou d'air.
- Après un déplacement horizontal, si la case en dessous est vide, vous **tombez** jusqu'à rencontrer de la roche ou le fond de la grotte.
- La distance de chute ne doit pas excéder F cases, sinon vous êtes blessé.
- Vous pouvez **creuser** une case de roche en diagonale basse (droite-bas ou gauche-bas), à condition que la case au-dessus de celle que vous creusez soit vide.
- Vous ne pouvez pas creuser en tombant.

Objectif : Atteindre le fond de la grotte en creusant le **minimum** de cases possible.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : R, C, F , puis la grille (. pour l'air, # pour la roche).

Sortie : Le nombre minimal de cases creusées, ou No si impossible.

20 Year of More Code Jam

Exercice 20 — Year of More Code Jam

Sphinny est passionnée de concours de programmation. Il y a T tournois cette année, et l'année compte N jours. Le tournoi i a R_i rounds, programmés aux jours $d_{i,1}, d_{i,2}, \dots, d_{i,R_i}$ **après** le début du tournoi.

Chaque tournoi commence un jour choisi **uniformément au hasard** parmi les N jours. Les choix sont indépendants, donc il y a N^T scénarios équiprobables.

Le **bonheur** de Sphinny se calcule ainsi : pour chaque jour de l'année, si S rounds (de tous les tournois confondus) ont lieu ce jour-là, le bonheur augmente de S^2 . Le bonheur initial est 0.

Note : Si un tournoi commence tard dans l'année, certains de ses rounds peuvent tomber l'année suivante et ne contribuent **pas** au bonheur.

Objectif : Calculer l'**espérance exacte** du bonheur, sous la forme $K + A/B$ où K et B sont des entiers positifs, A est un entier non-négatif, $0 \leq A < B$, et $\gcd(A, B) = 1$ si $A > 0$ (si $A = 0$, alors $B = 1$).

Format d'entrée / sortie

Entrée : T_{cases} cas. Chaque cas : N , T , puis pour chaque tournoi le nombre de rounds et les décalages en jours.

Sortie : L'espérance exacte sous la forme $K + A/B$.

Bon courage et bonne programmation !